

Construindo uma API REST Profissional com Node.js, TypeScript e Prisma

Introdução

Este documento registra as decisões técnicas e arquiteturais adotadas durante a construção de uma API REST para o **Sistema de Gestão de Treinamentos** da FG.

Mais do que desenvolver uma API funcional, o objetivo deste projeto é demonstrar como construir uma aplicação preparada para evoluir ao longo dos anos, mantendo organização, qualidade de código, facilidade de manutenção e baixo acoplamento.

Durante todo o desenvolvimento, cada decisão será discutida e justificada. O foco não será apenas "fazer funcionar", mas compreender por que determinada abordagem foi escolhida e quais benefícios ela traz para um projeto real.

O domínio do sistema foi definido previamente por meio da documentação funcional, que estabelece o objetivo da solução, os módulos do sistema, as regras de negócio, os requisitos funcionais e não funcionais, o modelo de dados e os fluxos operacionais. Essa documentação servirá como referência para todo o desenvolvimento da aplicação

Objetivos do Projeto

Este projeto possui dois objetivos principais.

O primeiro é construir uma API REST profissional para atender às necessidades do Sistema de Gestão de Treinamentos.

O segundo é utilizar esse desenvolvimento como ambiente de aprendizagem para demonstrar boas práticas utilizadas no mercado de desenvolvimento de software.

Ao final do projeto, espera-se que o aluno compreenda não apenas como utilizar determinado framework ou biblioteca, mas como organizar um sistema de forma consistente e sustentável.

Tecnologias Escolhidas

A escolha das tecnologias foi baseada em critérios como produtividade, robustez, comunidade e facilidade de manutenção.

Linguagem

TypeScript

Foi escolhido para adicionar tipagem estática ao JavaScript, reduzindo erros durante o desenvolvimento e melhorando a experiência de manutenção.

Framework

Express

Foi escolhido por sua simplicidade e maturidade no ecossistema Node.js.

Banco de Dados

MySQL

Banco relacional consolidado, amplamente utilizado no mercado e perfeitamente adequado ao domínio do sistema.

ORM

Prisma

Responsável por realizar a comunicação entre a aplicação e o banco de dados.

Apesar disso, o Prisma não será utilizado diretamente pelos Services. Ele ficará encapsulado na camada de Repository.

Validação

Zod

Toda validação da aplicação será realizada utilizando Zod.

Além disso, o Zod será considerado a única fonte da verdade para definição dos contratos da API.

Documentação

OpenAPI + Swagger UI

A documentação será gerada automaticamente a partir dos Schemas do Zod.

O fluxo adotado será:

Zod

↓

OpenAPI

↓

Swagger UI

Isso evita duplicação de código e garante que a documentação permaneça sincronizada com a implementação.

Testes

Vitest

Os testes fazem parte da arquitetura da aplicação.

Cada nova funcionalidade deverá ser acompanhada de seus respectivos testes.

Não será adotado TDD (Test-Driven Development) obrigatoriamente, mas sim uma cultura de testes contínuos.

Logs

Pino

Foi escolhido por oferecer excelente desempenho e geração de logs estruturados.

Esses logs serão fundamentais para auditoria, diagnóstico de problemas e monitoramento da aplicação.

Filosofia Arquitetural

O projeto adota os princípios do Domain-Driven Design (DDD) de forma pragmática.

Não será implementado o DDD completo, mas seus conceitos fundamentais orientarão toda a arquitetura.

Isso significa que:

- o domínio será mais importante que o banco de dados;
- os casos de uso serão mais importantes que operações CRUD;
- a linguagem utilizada no código refletirá a linguagem do negócio (ex: criarTreinamento()).

Organização do Projeto

A aplicação será organizada em módulos independentes.

Cada módulo representará uma responsabilidade específica do domínio.

Exemplo:

modules/

auth/

usuarios/

funcionarios/

instrutores/

treinamentos/

Essa abordagem reduz o acoplamento entre módulos e facilita futuras evoluções.

Estrutura em Camadas

A aplicação seguirá a seguinte organização:

Controller

↓

Service

↓

Repository

↓

Prisma

↓

Banco de Dados

Cada camada possui uma responsabilidade claramente definida.

Controller

Recebe a requisição HTTP.

Responsabilidades:

- receber parâmetros;
- validar entrada;
- chamar o Service;
- retornar a resposta.

Controllers não conterão regra de negócio (ex: não verifica de usuário já existe).

Service

Representa um caso de uso do sistema.

Toda regra de negócio ficará concentrada nesta camada.

Exemplos:

- criar usuário;
- autenticar usuário;

- iniciar treinamento;
- encerrar treinamento.

Repository

Encapsula toda comunicação com o Prisma.

Nenhum Service poderá acessar o Prisma diretamente.

Essa decisão reduz o acoplamento e facilita testes automatizados.

DDD Light (*Domain-Driven Design* ou Design Orientado a Domínio)

O projeto segue uma abordagem conhecida como DDD Light.

Foram adotados:

- linguagem ubíqua (UserTable -> Cliente, UserUpdatePass -> alterarSenha);
- organização por domínio; (Funcionários, Treinamentos)
- casos de uso (Funcionário->ações possíveis);
- separação de responsabilidades.

Não foram adotados:

- Aggregates;
- Value Objects;
- Domain Events;
- Factories complexas.

Essa decisão busca equilibrar qualidade arquitetural e produtividade.

Convenções de Código

Durante todo o projeto serão seguidas convenções padronizadas.

Pastas

Sempre em kebab-case.

Exemplo:

create-user

Arquivos

Sempre em kebab-case.

Exemplo:

usuario.controller.ts

usuario.schema.ts

create-user.service.ts

Classes

Sempre em PascalCase.

Exemplo:

CreateUserService

Funções

Sempre em camelCase.

Exemplo:

createUser()

Constantes

Sempre em UPPER_SNAKE_CASE.

TypeScript em modo Strict (Rigoroso)

O projeto utiliza o TypeScript com diversas verificações de segurança habilitadas.

Entre elas:

- `strict`: Ativa o modo rigoroso do TypeScript.;
- `exactOptionalPropertyTypes`: Diferencia uma propriedade opcional de uma propriedade cujo valor é `undefined`, evitando ambiguidades;
- `noUncheckedIndexedAccess`: Obriga a tratar valores possivelmente `undefined` ao acessar arrays ou objetos por índice/chave.;
- `noImplicitOverride`: Exige o uso da palavra-chave `override` ao sobrescrever métodos de uma classe pai;
- `noFallthroughCasesInSwitch`: Impede que um case do switch continue executando o próximo case sem um `break`, `return` ou `throw`.

Essas configurações permitem detectar erros ainda durante a compilação.

Variáveis de Ambiente

Todas as configurações da aplicação são centralizadas em um único arquivo.

src/config/env.ts

Nenhuma outra parte da aplicação acessará process.env diretamente.

Além disso, todas as variáveis são validadas pelo Zod no momento da inicialização da aplicação.

Caso alguma variável obrigatória esteja ausente ou inválida, a aplicação será encerrada imediatamente.

Logger

Foi adotada a biblioteca Pino.

A configuração foi organizada em módulos independentes.

core/

logger/

index.ts

options.ts

Essa organização separa a configuração da criação da instância do logger.

Sempre que possível, os logs deverão ser estruturados.

Exemplo:

```
logger.info(  
  {  
    port,  
    environment,  
  },  
  "Servidor iniciado."  
);
```

Logs estruturados facilitam consultas futuras e integração com ferramentas de observabilidade.

Validação com Zod

Toda entrada de dados será validada utilizando Zod.

Além disso, os próprios Schemas gerarão os tipos TypeScript.

Exemplo:

```
export const createUserSchema = ...
```

```
export type CreateUserInput =  
  z.infer<typeof createUserSchema>;
```

Com isso, elimina-se a necessidade de criar DTOs separados.

Documentação Automática

A documentação da API será gerada automaticamente.

Fluxo:

Schema Zod

↓

OpenAPI

↓

Swagger UI

Assim, sempre que um Schema for alterado, a documentação será atualizada automaticamente.

Testes

Os testes fazem parte do desenvolvimento.

O fluxo adotado será:

1. Implementar a funcionalidade.
2. Criar ou atualizar os testes (Criar um agente de IA para fazer isso).
3. Executar a suíte de testes.
4. Considerar a tarefa concluída.

Essa prática reduz regressões e aumenta a confiabilidade da aplicação.

Princípios Arquiteturais

Durante todo o desenvolvimento foram definidos alguns princípios que deverão orientar futuras decisões.

1. Não criar abstrações antes da necessidade

Classes, interfaces, utilitários e pastas somente serão criados quando houver necessidade real.

Evita complexidade desnecessária.

2. Um arquivo deve possuir apenas uma responsabilidade

Cada arquivo deverá possuir um único propósito claramente definido.

3. A infraestrutura não conhece o domínio

Componentes como Logger, Prisma, OpenAPI e Middlewares pertencem à infraestrutura.

Eles não devem conhecer regras de negócio.

4. O domínio dirige a arquitetura

As regras de negócio definem a estrutura do sistema.

O banco de dados não determina como a aplicação será organizada.

5. Zod é a fonte única da verdade

Não haverá duplicação de definições entre validação, tipos TypeScript e documentação.

6. Controllers devem ser extremamente simples

Toda lógica de negócio ficará concentrada nos Services.

7. Repositories encapsulam a persistência

A camada de domínio nunca dependerá diretamente do Prisma.

8. Testes fazem parte da arquitetura

Uma funcionalidade somente será considerada concluída quando possuir testes correspondentes.

Estado Atual do Projeto

Até o momento foram concluídas as seguintes etapas:

- definição da arquitetura geral;
- escolha da stack tecnológica;
- configuração do TypeScript;
- configuração do ESLint;
- configuração do Prettier;
- configuração das variáveis de ambiente;
- configuração inicial do Prisma;
- configuração do Logger com Pino;
- configuração do ambiente de desenvolvimento em WSL;
- definição da estratégia de documentação utilizando Zod + OpenAPI + Swagger UI;
- definição da estrutura modular da aplicação.

Próximas Etapas

A infraestrutura continuará sendo construída antes da implementação dos módulos de negócio.

A sequência prevista é:

1. Tratamento global de erros.
2. Respostas padronizadas da API.
3. Configuração definitiva do Prisma.
4. Configuração do OpenAPI.
5. Implementação do módulo de Autenticação.
6. Implementação dos módulos de Usuários, Funcionários, Instrutores e Treinamentos.

Conclusão

Este projeto tem como objetivo demonstrar que desenvolver software profissional vai muito além de escrever código.

Uma boa arquitetura é construída por meio de decisões conscientes, organização consistente e respeito aos princípios de engenharia de software.

Ao longo do desenvolvimento, cada tecnologia utilizada será apresentada não apenas sob o aspecto técnico, mas também dentro do contexto arquitetural em que ela agrega valor.

O resultado esperado não é apenas uma API funcionando, mas uma base sólida que possa crescer, ser mantida por diferentes equipes e evoluir com segurança ao longo do tempo.